ACTIVIDAD 8: PROGRAMACIÓN ORIENTADA A OBJETOS (POO) EN C++

JUEGO DE NAVES EN CONSOLA (class Nave)

Informe individual

Laboratorio de Programación (LPR) — 5° 3° Grupos A-B

E.E.S.T. N.° 1 "Eduardo Ader" — Vicente López

Estudiante (Nombre y Apellido): Teo Laguna

Curso / Grupo: 5° 3° Grupos A / B

Materia: Laboratorio de Programación

Profesor: Mansilla Muñoz York Elías y Yamil Ganduglia

Repositorio: juegodenaves/ (src/, docs/, docs/manuales/, capturas/)

Link del repositorio: https://github.com/teolaguna/juegodenaves.git

Link del Gitlab: https://gitlab.com/teolaguna/juegonaves.git

Modalidad / Versión: Individual v1.0.0

Fecha de entrega: 21 de octubre de 2026

Resumen

Se desarrolló de forma individual un simulador de combate espacial en consola aplicando POO en C++. Se definieron las clases Nave, Asteroide y Bala con atributos privados, constructores y métodos públicos, respetando el encapsulamiento. El programa compila con g++ -std=c++11 y fue verificado en Windows. Se integró la funcionalidad 1 (Score), que suma puntos al destruir asteroides y habilita el disparo doble a partir de 300 puntos.

Palabras clave: C++, POO, class, private, public, constructor, encapsulamiento, Score, juego de naves.

1. Introducción

En la Actividad 7 se usó struct para agrupar variables; la POO da el siguiente paso al reunir, en una clase, los datos (atributos) y las funciones que los modifican (métodos). En C++ la única diferencia técnica entre struct y class es el acceso por defecto: público en struct y privado en class. Esta actividad construyó un juego de naves en consola definiendo la clase Nave con atributos privados, métodos públicos y encapsulamiento, y documentó el software según las normas APA.

2. Desarrollo

El motor se organizó en tres clases. Nave modela al jugador con atributos privados x, y y vidas, constructor Nave(int, int, int) y métodos getX, getY, getVidas, reducirVida, pintar, borrar y mover. Asteroide y Bala representan los obstáculos que caen y los disparos. Al declarar los atributos en private:, solo los métodos públicos de la clase pueden modificarlos, lo que evita valores inválidos. En el main() se crea el jugador, se inicializan los vectores y el bucle principal gestiona el HUD, el teclado y las colisiones hasta el Game Over.

Cuestionario Técnico de Control

Pregunta 1 struct vs class: diferencia técnica respecto de private y public.

Respuesta: la única diferencia es el acceso por defecto; struct es público y class es privado. Ambos admiten métodos y constructores. Aquí se declararon explícitamente private: (datos) y public: (acciones).

Pregunta 2 Encapsulamiento: ¿por qué es peligroso dejar la salud pública?

Respuesta: permitiría asignar valores inválidos desde el main() (p. ej. vidas negativas o exceso), rompiendo la lógica. El encapsulamiento obliga a usar el método controlado reducirVida, manteniendo el estado siempre válido.

Pregunta 3 Constructor: ¿para qué sirve Nave(int _x, int _y, int _vidas)?

Respuesta: se ejecuta al crear el objeto e inicializa los atributos privados con los valores recibidos, dejando el objeto en un estado válido desde su creación.

Pregunta 4 Funcionalidad elegida: ¿cuál y dónde la insertaste?

Respuesta: funcionalidad 1 (Score). Agregué int score = 0; y su línea en el HUD, y score += 10; en la colisión bala-asteroide. Modifiqué el if (tecla == ' ') para que, si score < 300, dispare una bala central y, si score >= 300, dos balas desde las alas. El total se muestra como SCORE FINAL en Game Over.

3. Conclusión

La POO organiza el código reuniendo datos y comportamiento en una clase; el encapsulamiento protege el estado interno y el constructor garantiza un inicio válido. La funcionalidad Score demostró cómo extender el motor sin alterar las clases. Documentar con informe APA y manuales confirmó que el software profesional también se documenta.